SUTD404: Denoising Language Models for Room-to-Address Lookup

G. Lim, E. Ang, M. Shaswat, M. Ramaswamy

April 2026

1 Introduction

The Singapore University of Technology and Design (SUTD) has over 200 rooms split across 5 buildings. Rooms are often communicated by name (e.g. "Think Tank 1"), and each room is assigned an address of the form

```
x.y-z
```

where `x` is a building number, `y` is a floor number, and `z` is a room number. Room-to-address lookup—mapping rooms to their addresses—can be a hassle for staff, students, and visitors alike. However, resources for this task remain limited. Physical resources like campus signs have been constructed, but often require one to expend energy and time walking around. Over the years, digital resources like the SUTD Assistive Maps (SAM) Telegram bot (Telegram). However, they often fail on noisy input (e.g. strings with typos), which can be especially common among visitors unfamiliar with the rooms and people with less experience typing in English.

To address this gap, we introduce SUTD404, a digital resource for noisy room-to-address lookup. Visit **github.com/gregorylimeurhen/sutd404** for our code repository, and **sutd404.vercel.app** for our web application.

2 Methodology

Let V be a vocabulary, and $R \subseteq V^*$ be a room set, and $\prec$ be a total order on R, and $N \in \mathbb{N}_{\geq 1}$ be a result count, and $\prec_N$ be a total order on $R^N_{\prec}$, and $A \subseteq V^*$ be an address set, and $B \subseteq V$ be a boundary set, and $C \in \mathbb{N}$ be a corruption count, and $\alpha \in \mathbb{R}_{[0,1]}$ be a transposition rate, and $\beta \in \mathbb{R}_{[0,1]}$ be a substitution rate, and $\boldsymbol{\pi} = (\pi_{\text{train}}, \pi_{\text{val}}, \pi_{\text{test}}) \in \Delta^2_C$ be a training-validation-testing split, and $I : V \rightharpoonup \mathbb{Z}^2$ be a keyboard-position map.

Definition 1 (Keyboard)**.**

$$K = \begin{bmatrix} \texttt{`} & \texttt{1} & \texttt{2} & \texttt{3} & \texttt{4} & \texttt{5} & \texttt{6} & \texttt{7} & \texttt{8} & \texttt{9} & \texttt{0} & \texttt{-} & \texttt{=} \\ \texttt{q} & \texttt{w} & \texttt{e} & \texttt{r} & \texttt{t} & \texttt{y} & \texttt{u} & \texttt{i} & \texttt{o} & \texttt{p} & \texttt{[} & \texttt{]} & \texttt{\backslash} \\ & \texttt{a} & \texttt{s} & \texttt{d} & \texttt{f} & \texttt{g} & \texttt{h} & \texttt{j} & \texttt{k} & \texttt{l} & \texttt{;} & \texttt{'} & \\ & \texttt{z} & \texttt{x} & \texttt{c} & \texttt{v} & \texttt{b} & \texttt{n} & \texttt{m} & \texttt{,} & \texttt{.} & \texttt{/} & & \\ & & & & & \texttt{␣} & \texttt{␣} & \texttt{␣} & \texttt{␣} & \texttt{␣} & & & \end{bmatrix}$$

Our keyboard uses the 60% ANSI QWERTY en-US keyboard layout without backspace, tab, caps lock, enter, shift, ctrl, alt, and system keys (Wikipedia contributors, 2026). All keys have equal padding between them.

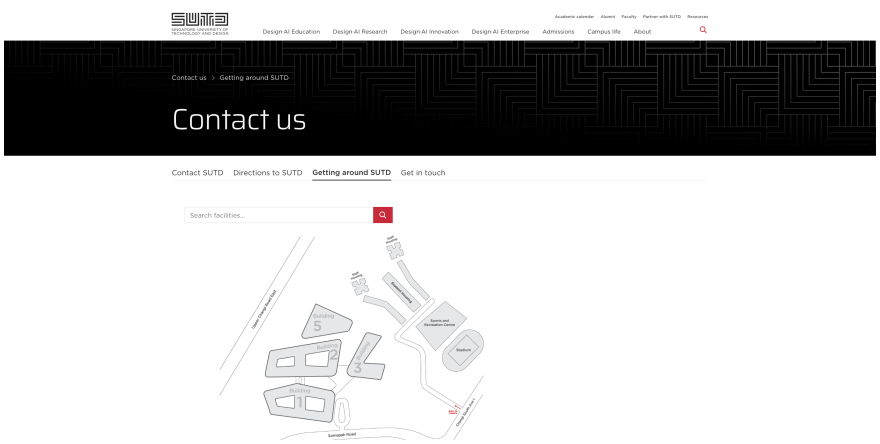

Figure 1: Official SUTD contact pages.

We obtained R by politely scraping official SUTD contact pages. The facility search function was queried by enumerating buildings and floors.

Algorithm 1 CORRUPT($\boldsymbol{x}$)

Require: $\boldsymbol{x} \in V^*$

$(T_L, T_M, T_R) \leftarrow (V^*B \cup \{()\}, (V \setminus B)^+, BV^* \cup \{()\})$

$T \leftarrow \{\boldsymbol{uwbvz} \mid \boldsymbol{u} \in T_L,\ \boldsymbol{v}, \boldsymbol{w} \in T_M,\ \boldsymbol{z} \in T_R,\ b \in B,\ \boldsymbol{x} = \boldsymbol{uvbwz}\} \setminus \{\boldsymbol{x}\}$

if $T \neq \varnothing$ **then** $\boldsymbol{x} \sim \alpha\,\mathcal{U}(T) + (1-\alpha)\,\delta(\boldsymbol{x})$

for $i \in [1..|\boldsymbol{x}|]$ **do**

  $S \leftarrow \{k \in K \setminus \{x_i\} \mid I(k) - I(x_i) \in [-1..1] \times [-2..2]\}$

  if $S \neq \varnothing$ **then** $x_i \sim \beta\,\mathcal{U}(S) + (1-\beta)\,\delta(x_i)$

return $\boldsymbol{x}$

Algorithm 2 SPLIT()

$(D_{\text{train}}, D_{\text{val}}, D_{\text{test}}) \leftarrow (\varnothing, \varnothing, \varnothing)$

for $r \in R$ **do**

  $P \leftarrow \varnothing$

  while $|P| < C$ **do**

    $r' \sim$ CORRUPT(r)

    if $r' \neq r$ **then** $P \leftarrow P \cup \{(r', r)\}$

  $P_{\text{train}} \sim \mathcal{U}(\{Q \subseteq P \mid |Q| = \pi_{\text{train}} C\})$

  $P_{\text{val}} \sim \mathcal{U}(\{Q \subseteq P \setminus P_{\text{train}} \mid |Q| = \pi_{\text{val}} C\})$

  $P_{\text{test}} \leftarrow P \setminus (P_{\text{train}} \cup P_{\text{val}})$

  $(D_{\text{train}}, D_{\text{val}}, D_{\text{test}}) \leftarrow (D_{\text{train}} \cup P_{\text{train}}, D_{\text{val}} \cup P_{\text{val}}, D_{\text{test}} \cup P_{\text{test}})$

return $(D_{\text{train}}, D_{\text{val}}, D_{\text{test}})$

We generated corrupted strings and split corrupted-uncorrupted room pairs into training, validation, and testing sets. Our corruption algorithm corrupts strings by transposing spans and substituting characters. First, the algorithm samples whether to transpose spans by uniformly sampling from the space of span transpositions. Then, for each character in the string, the algorithm samples whether to substitute the character by uniformly sampling from the space of character substitutions. This space contains nearby characters collected by sliding a window of height 3 and width 5 centred at each position in K. Finally, the algorithm returns the string. We set $C = 50$, and $\alpha = 0.5$, and $\beta = 0.05$, and $\boldsymbol{\pi} = (0.8, 0.1, 0.1)$ to generate plausible-looking typos in a tractable way. The split is stratified by room to make the training, validation, and test sets comparable across rooms.

Definition 2 (Denoiser)**.** Let $f : V^* \times R \to \mathbb{R}$ be a scorer.

$$\mathscr{D}_f : V^* \to R$$
$$\mathscr{D}_f(x) = \min_{\prec} \arg\max_{r \in R} f(x, r)$$

Our denoiser is a Nanochat-parameterised, character-level transformer of depth 2 with decoding constrained to the trie induced by R (Karpathy, 2025). We use character-level tokenisation as typos are often character-level, and room names are often short enough for individual characters to be meaningful lexical units for our task. We use trie-constrained decoding to efficiently avoid room hallucination.

Our denoiser is trained using a standard cross-entropy loss and AdamW optimiser. A more complete list of hyperparameters can be found in our code repository.

Definition 3 (Lookup)**.**

$$\mathscr{L} : R \to A$$

Our lookup is a standard room-address table.

Algorithm 3 INFER(f, x)

Require: $f : V^* \times R \to \mathbb{R}$, $x \in V^*$
  if $x \notin R$ **then** $x \leftarrow \mathscr{D}_f(x)$
  $x \leftarrow \mathscr{L}(x)$
  return x

Our inference algorithm receives a scorer and string as input. Firstly, the algorithm denoises the string to a room using the scorer if the string is not a room. Then, the algorithm looks up the address of the room. Finally, the algorithm returns the address.

Denoiser	**Accuracy** (%)	**Mean Latency** (ms)
LCP	29.2	**0.04**
DL	71.9	16.7
SJ	89.3	20.8
Ours	**97.3**	8.19

Table 1: Denoiser evaluation results summary.

We first evaluated our denoiser against substring Jaccard similarity (SJ), negative Damerau–Levenshtein distance (DL), and longest common prefix length (LCP) denoisers, measuring the accuracy and mean latency of each denoiser on the test set. To maximise fairness and reproducibility, we fixed a seed and hardware. Accuracy was measured using the fraction of test inputs for which final address equals gold address. Mean latency was measured using the mean time taken to get an output from giving an input.

Algorithm 4 $\textsc{Predict}(x)$

Require: $x \in V^*$
  $x \leftarrow \textsc{Strip}(x)$
  $x \leftarrow \textsc{Depunctuate}(x)$
  $x \leftarrow \textsc{Lowercase}(x)$
  $x \leftarrow \textsc{AlphanumericSplit}(x)$
  $x \leftarrow \textsc{Alias}(x)$
  if $x \in R$ **then return** $((x, \mathscr{L}(x)))$
  $\boldsymbol{f} \leftarrow (\mathrm{LCP}, \mathrm{SJ}, \mathrm{DL}, \textsc{Ours})$
  for $r \in R$ **do**
    for $f \in \boldsymbol{f}$ **do**
      $p(r, f) \leftarrow |\{r' \in R \mid f(x, r') > f(x, r)\}| + 1$
  for $r \in R$ **do**
    $s(r) \leftarrow \sum_{i=1}^{|\boldsymbol{f}|} (1/|R|^{i-1})(|R| - p(r, f_i))$
  $\boldsymbol{o} \leftarrow \min_{\prec_N} \arg\max_{\boldsymbol{r} \in R_{\succ}^N} \sum_{i=1}^{N} s(r_i)$
  return $((o, \mathscr{L}(o)) \mid o \in \boldsymbol{o})$

We implemented and deployed our system as a web application. We set $N = 26$ because, grouping rooms by shared prefix up to terminal numbers, the largest group (the group of Think Tank rooms) contains 26 rooms, so showing 26 results could allow a user to find their room by prefix without disambiguating by terminal numbers. Our prediction algorithm receives a string as input. Firstly, the algorithm normalises the string by stripping whitespace, removing punctuation, lowercasing, separating letters from numbers, and replacing words with their aliases, to make the string more closely match training strings. Then, the algorithm computes top-ranked rooms using harmonically-weighted Borda

ranking. Finally, the algorithm looks up the address of each room, returning top-ranked room-address pairs.

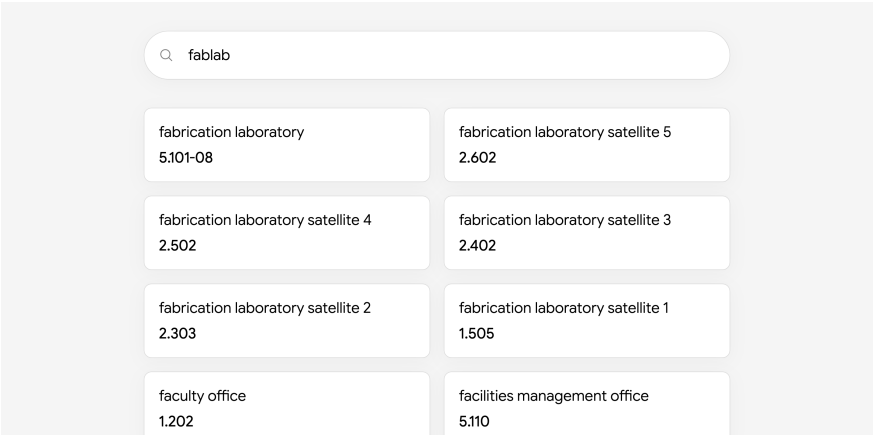

Figure 2: Web application.

To further improve the user experience, we also implemented the following features to make our web application more efficient and responsive.

- **Incremental prediction.** We predict during typing rather than waiting for final submission.
- **Prefix caching.** We KV-cache previously computed prefix states and only process newly appended characters.
- **Debounced querying.** We introduce a short debounce to avoid redundant intermediate inference on rapid keystrokes.
- **Redundant-query suppression.** We skip empty, already displayed, and exact-match queries when full recomputation is unnecessary.
- **Bounded result caching.** We memoise recent solved queries while capping cache growth to prevent performance degradation over time.

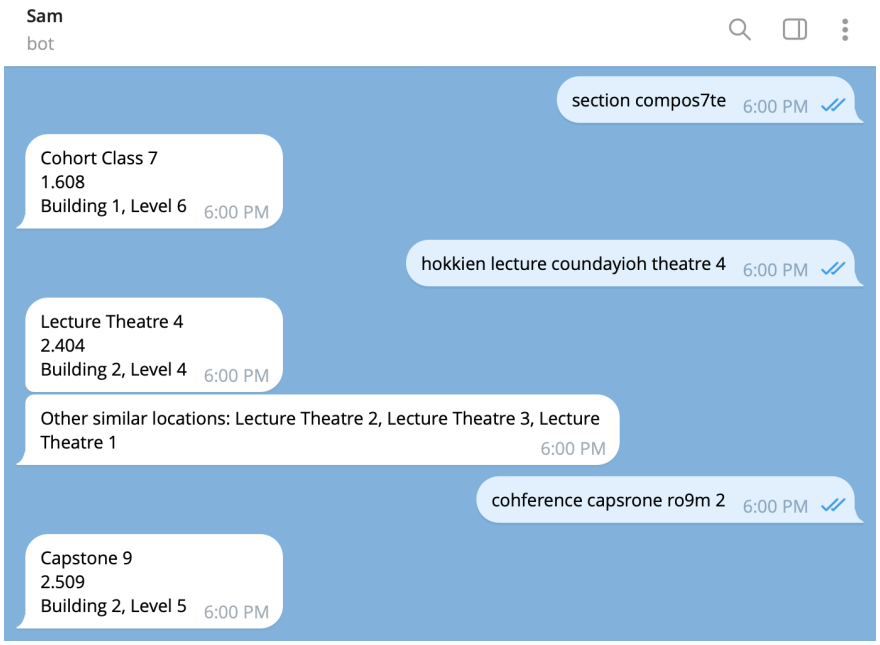

Figure 3: SAM.

We further evaluated our system against SAM, measuring accuracy on a held-out set of 10 inputs sampled from our test set using the Python module `random` with UTF-8 encoding. To maximise fairness and reproducibility, we fixed the same seed and hardware as before.

Input	design bdhavioural laboratory 1
Gold	behavioural design laboratory 1
Prediction	–
Input	capstone rolm 8
Gold	capstone room 8
Prediction	capstone room 8, capstone room 7, capstone room 6, capstone room 5
Input	lecturr 5 theatre
Gold	lecture theatre 5
Prediction	lecture theatre 5, lecture theatre 4, lecture theatre 2, lecture theatre 3
Input	com;ositemdesign laboratory
Gold	composite design laboratory
Prediction	–
Input	thinl tank 23
Gold	think tank 23
Prediction	think tank 15, think tank 24, a c toh think tank 25, think tank 22
Input	temasekvlaboratories and itrust labiratkry 6
Gold	temasek laboratories and itrust laboratory 6
Prediction	temasek laboratories and itrust laboratory 6
Input	think 2mtank
Gold	think tank 2
Prediction	think tank 2, think tank 14
Input	section compos7te
Gold	composite section
Prediction	cohort classroom 7
Input	hokkien lecture coundayioh theatre 4
Gold	hokkien foundation lecture theatre 4
Prediction	hokkien foundation lecture theatre 4, lecture theatre 2, lecture theatre 3, lecture theatre 1
Input	cohference capsrone ro9m 2
Gold	capstone conference room 2
Prediction	capstone room 9

Table 2: SAM evaluation results.

Our results suggest that SAM, as a traditional matching-based algorithm, can be effective on mildly corrupted inputs with strong lexical overlap—occasionally recovering the exact target—but degrades markedly as noise becomes more severe. We conclude that SAM can be robust to limited orthographic variation, but can also fail to generalise beyond surface-form similarity.

System	**Accuracy** (%)
SAM	40
Ours	**100**

Table 3: System evaluation results summary.

Our system outperformed SAM, achieving complete recovery. We believe these results demonstrate strong promise and provide practical foundations for reliable campus assistance.

3 Appendix

Notation 1. Let $n \in \mathbb{N}_{\geq 1}$.

$$R^n_{\prec} = \{\boldsymbol{r} \in R^n \mid r_1 \prec \cdots \prec r_n\}$$

Notation 2. Let $c \in \mathbb{N}$.

$$\Delta^2_c = \left\{ \boldsymbol{p} \in \mathbb{R}^3_{[0,1]} \;\middle|\; \sum_{i=1}^{3} p_i = 1,\, c\boldsymbol{p} \in \mathbb{N}^3 \right\}$$

References

Andrej Karpathy. nanochat: The best chatgpt that $100 can buy. `https://github.com/karpathy/nanochat`, 2025.

Telegram. Telegram: Launch @sutdmapbot. URL `https://t.me/SUTDMapBot`.

Wikipedia contributors. Qwerty. `https://en.wikipedia.org/wiki/QWERTY`, 2026.